

Worksheet 1: SIMD

In this worksheet, you will begin parallelizing our heat equation solver. Although we will remain on a single CPU core, we will exploit the CPU's Single Instruction, Multiple Data (SIMD) capabilities to accelerate the implementation by leveraging data-level parallelism. To achieve this, modifications are required in three key components of the codebase: the vector implementation, the matrix implementation, and the heat operator.

This worksheet will guide you through the necessary changes step by step and explain how to use the provided code framework to evaluate the performance impact of your optimizations. By the end, you will gain a deeper understanding of how vectorization can significantly improve computational efficiency — even without resorting to multi-threading or multi-core execution.

Preliminaries

Before we dive into the code, you need to ensure that the docker container is properly set up and running. If you haven't done this yet, please follow the instructions provided in the course materials to get everything up and running.

Getting Started

Once opening the project folder in VSCode you should be prompted to reopen it in the container. If not, you can do this manually by clicking on the green icon in the bottom left corner of VSCode and selecting "Reopen in Container". After a short moment the container environment should be ready. Please note, that in case you have not yet setup the container image or anything changed in the docker file this might take a bit longer.

You find detailed installation instructions in the README provided in the project's root folder. If you run into any trouble, just ask — we are happy to help!

Code Structure

The project consists of five main folders:

- **applications/** – Contains the executable for the heat equation solver. This is not relevant for this week, but we will be working with it in the upcoming exercises. Feel free to add applications if you want to experiment with your own code.
- **include/i2php/** – Header files for your classes and logic.
- **source/** – The corresponding implementation (.cpp) files.
- **benchmarks/** – Contains benchmarks to evaluate the performance of your implementations.

- **tests/** – Contains unit tests which you can run to verify the correctness of your implementations. See the README for more details. Also feel free to add your own tests. Writing tests is a great way to deepen your understanding of the code. You might wanna have a look at test-driven development and GoogleTest for this.

Submission

Please submit your solution until Wednesday, June 10th, 2026 11:59 PM by opening a merge request of your developing branch into the main branch of the repository. The merge request should include the following:

- The modified code files including all required changes for the worksheet.
- A short PDF document summarizing the performance analysis of your implementations, including the benchmark results and your reasoning about the observed trends. In addition, include the answers to the guiding questions provided in the worksheet, along with any relevant observations or insights you gained during the implementation and testing process.

Today's Worksheet Tasks

As mentioned earlier, today's worksheet focuses on vectorizing our implementation of the heat equation solver. Specifically, the primary objective is to vectorize the **vector** and **matrix** classes before moving on to the solver itself.

You will begin with the vector class, followed by the matrix class. While you are free to work on the tasks in any order, we highly recommend starting with the vector implementation. It is not only the simpler of the two, but the worksheet also provides more detailed guidance for the vector class compared to the matrix class.

Once you have completed the vector implementation, adapting the matrix class should be relatively straightforward, as many of the concepts and techniques can be reused. However, there is an important distinction when implementing the matrix-matrix product. Don't worry—the worksheet will walk you through the required modifications.

Throughout the tasks, you will be asked at specific points to perform performance measurements using the provided benchmarking framework. Please summarize your findings in a short PDF document. It is essential that you include the actual measurements you used as the basis for your analysis so that your reasoning is clear and transparent during grading. In addition, please include either the vectorization width of your CPU or the exact CPU model to help contextualize the performance results.

With that said, enjoy working through the worksheet!

Task 1: Linear Algebra Vector (20 Points)

In this task, we focus on one of the fundamental components of our heat equation solver: the linear algebra vector class. Our goal is to extend its functionality by incorporating

support for SIMD (Single Instruction, Multiple Data) vectorization. Before delving into the implementation details, we first take a look at the changes that have been made to the code skeleton compared to previous weeks.

The most notable modification is that the class now includes an additional template parameter and is defined as follows:

```
template <typename number, int N = 1>
class i2phpp::LaVector;
```

As introduced in earlier exercise sheets, the template parameter `number` specifies the primitive data type used for storing the vector's elements, such as `float` or `double`. The second template parameter `N` is new and determines the vectorization width—that is, the number of elements processed simultaneously during SIMD operations. A default value of 1 is provided to ensure that the code compiles even if vectorization has not yet been implemented.

In practice, this kind of parameter is typically not passed explicitly. Instead, developers often rely on abstraction layers that automatically use the appropriate vector width based on the target hardware. However, to allow for experimentation with different widths and to enable performance analysis, we deliberately expose this parameter here. An alternative approach would involve passing a SIMD type directly—one that already encodes both the data type and the vector width—but this is avoided for simplicity in this exercise.

Another important change is that the class has been converted into a header-only implementation. This means that both the declarations and the definitions of all functions now reside within the same header file. Typically, the function declarations, along with accompanying documentation, are located at the top of the file, while the actual function bodies are implemented at the bottom. This structure is particularly useful for template-based development because it removes the need to explicitly instantiate the code for particular template parameters at compile time.

For testing, you should continue using the test suite provided in `tests/la_vector_tests.cpp`. Initially, all tests will still pass, even if you haven't modified the implementation, since the original scalar version remains valid. However, once vectorization is introduced, the test cases should help uncover most implementation errors or unexpected behaviors.

With that, we have covered all the relevant updates to the code base for this task. We now turn to the subtasks of this exercise.

Vectorization Without Remainder Vectorization

As a first step, we will vectorize the implementation of the linear algebra vector operations—while temporarily ignoring the remainder elements that do not fit evenly into SIMD vector registers. This allows us to focus on the main structure and logic of vectorized computations before dealing with edge cases.

The following member functions of the `LaVector` class must be adapted to use SIMD operations:

- `number dot_product(const LaVector<number, N>& other) const;`
- `LaVector<number, N> operator+(const LaVector<number, N>& vec) const;`
- `LaVector<number, N>& operator+=(const LaVector<number, N>& vec);`
- `LaVector<number, N> operator-(const LaVector<number, N>& vec) const;`
- `LaVector<number, N>& operator-=(const LaVector<number, N>& vec);`
- `LaVector<number, N> operator*(number num) const;`
- `LaVector<number, N>& operator*=(number num);`
- `LaVector<number, N> operator/(number num) const;`
- `LaVector<number, N>& operator/=(number num);`
- `number l2_norm_squared() const;`
- `number norm_l2() const;`
- `number norm_l1() const;`
- `number norm_linf() const;`
- `LaVector<number, N> schur_product(const LaVector<number, N>& other) const;`

To vectorize these functions, the main loop that processes the elements must be rewritten to operate in chunks of N elements at a time, corresponding to the chosen vectorization width. This involves updating both the loop's increment step and its upper bound. It is important to ensure that the upper bound is chosen such that the SIMD operations do not attempt to access memory beyond the actual size of the data. Within the loop body, you will load the relevant data from the vectors into SIMD registers, perform the vectorized arithmetic operation, and then store the result back into the appropriate memory location.

At this point, we intentionally leave out the proper vectorized handling of the remainder elements—that is, the final few elements of the vector that do not fill a complete SIMD register. These can instead be handled with a simple scalar loop following the vectorized loop. This scalar loop processes any remaining elements one by one and ensures that the entire vector is correctly computed, even if it is not a perfect multiple of N .

This initial vectorization step lays the groundwork for improved performance while keeping the implementation relatively simple. Full remainder vectorization will be addressed in a later subtask.

Note: You should assume that the data is **not** aligned in memory according to the vectorization width of the SIMD vector.

Pitfall When Using Incorrect Loop Bounds

While implementing your vectorized loop, you may have noticed something unexpected: even if you use incorrect loop bounds—for example, if the upper bound is too large and you load data beyond the valid range of the vector—the code might still run without crashing, and the results may even appear to be correct. If you identified and used the correct loop bounds from the start, excellent! However, if you did not and things still seemed to work, you might be wondering: why does this happen? And more importantly, could this behavior be used as a shortcut for handling the remainder elements?

We intentionally leave this as a question for you to explore. Try to reason about the memory layout of vectors, how SIMD loads work, and why reading slightly beyond the end might not always lead to an error. Then, consider why this is still not a reliable or safe approach for dealing with the remainder in a general and portable way.

A few guiding questions to consider:

- What does your system typically do when accessing memory just beyond the end of an array?
- How does `std::vector` allocate memory?

Reflect on these points and try to formulate a short explanation based on your observations and testing.

Vectorizing the Remainder

The final step in this task is to extend the vectorization to also cover the remainder — that is, the few elements at the end of a vector that do not fill an entire SIMD register. While for vectors with thousands of elements the performance gain from vectorizing a handful of remaining values may be negligible, our goal here is to be precise and complete in our implementation.

So far, the remainder has been handled using a scalar loop. Now, we want to eliminate that scalar fallback and instead implement a masked SIMD operation that handles the remaining elements in a fully vectorized manner. To do this, you will need to use conditional (masked) SIMD operations as introduced in the lecture or script. In particular, you can make use of the `std::experimental::where()` function to selectively operate only on valid entries of the final SIMD register. This requires the construction of an appropriate SIMD mask that marks which elements should be processed and which should be ignored.

Once the correct elements are conditionally loaded into a SIMD vector, you can apply the usual vectorized operation (e.g., addition, subtraction, multiplication, etc.) and then use the same mask to conditionally store the result back into the corresponding positions of the vector.

Analyzing the Performance

Great job! You have now successfully vectorized our implementation of the linear algebra vector class. It's time to assess the impact of your work by analyzing performance

improvements achieved through SIMD vectorization. For this purpose, a set of benchmarks is provided in the file `benchmarks/linear_algebra/la_vector_benchmark.cpp`. Instructions for running these benchmarks can be found in the project's README.

The benchmark output printed to the console provides several pieces of valuable information:

- **Benchmark Name:** Indicates the function being benchmarked, along with the template parameters used—most notably, the vectorization width. For example:

```
BM_VectorAddition<data_type, 4>
```

This tells us that the benchmark is testing vector addition using a vectorization width of 4. The data type `data_type` used in this case is defined within the benchmark file via:

```
/// The data type stored in the vectors used for benchmarking.  
using data_type = float;
```

You are encouraged to experiment with this alias and observe how different data types (e.g., `double` or `int`) affect performance.

- **Time:** The wall-clock time recorded for the benchmark execution.
- **CPU:** The CPU time spent executing the benchmark. This is the most relevant metric for comparing performance across different vectorization widths.
- **Iterations:** The number of repetitions automatically performed until statistically stable results were achieved.

Your task is to analyze the performance output printed to the console after running the benchmarks. Focus particularly on the **CPU time**, which reflects the actual processing time spent by the CPU and is the most meaningful metric for comparing the efficiency of different vectorization configurations when neglecting memory effects. Experiment with a range of vectorization widths, data types, and vector sizes. Based on the results, reflect on and answer the following guiding questions in your analysis:

- What performance trends can you observe as the vectorization width increases? Does a wider SIMD width always lead to better performance?
- Are there diminishing returns at higher widths? At what point does increasing the width no longer yield significant speed-ups?
- How does the choice of data type (`float`, `double`, `int`, etc.) influence performance?

Describe the trends you observe during testing and provide possible explanations for them. It is important to note that, due to the experimental nature of the `std::experimental::simd` compiler implementation, the speedups you observe may not always match your expectations. That is completely fine and not a problem! The primary goal of this exercise is to

identify and analyze performance trends, reason about their potential causes and discuss the differences of the performance results to your expectations.

If you are particularly interested in exploring additional performance effects, you may also experiment with different vector sizes in the benchmark. This can be done by modifying the following line near the beginning of the benchmark file:

```
/// The length of a vector used for benchmarking.  
constexpr std::size_t vector_size = 73;
```

Changing this value allows you to investigate how the total size of the vector impacts performance under various vectorization widths and data types. In particular, it is interesting to compare benchmark runs with very small versus very large vector sizes.

Note: In your report, please include the complete benchmark output from your local machine, including the data types used and the corresponding CPU times. Additionally, make sure to indicate the **vectorization width (in bits)** supported by your CPU. This information is crucial for interpreting your results and understanding the possible limitations of your hardware.

Task 2: Linear Algebra Matrix (20 + 2 Points)

Now that we've vectorized our linear algebra vector implementation, let's move on to the matrix implementation.

First, let's discuss the updates made to the codebase compared to the version we've used in the past few weeks. Just like with the vector class, we introduce a new template parameter, `N`, which defines the vectorization width. Accordingly, the matrix class `LaMatrix` is now declared as:

```
template <typename number, int N = 1>  
class i2php::LaMatrix;
```

Additionally, the entire implementation has been moved to a header-only library. This means that both the declarations and definitions of the member functions can now be found in the corresponding header file, similar to the vector implementation.

You can reuse the tests from the scalar version of the matrix class. These tests have been slightly adapted to check for additional edge cases that might arise during vectorization. If you haven't modified anything at the beginning of your implementation, these tests should still pass, since the scalar implementation is preserved in the project skeleton.

Vectorization of the Main Methods

In this subtask, your goal is to vectorize the matrix class, with the exception of the following two functions:

- `number trace() const;`
- `LaMatrix<number, N> operator*(const LaMatrix<number, N> &mat) const;`

These implementations come with some extra challenges and will therefore be handled later in the task.

For now, focus on vectorizing the following member functions:

- `LaMatrix<number, N> operator+(const LaMatrix<number, N> &mat) const;`
- `void operator+=(const LaMatrix<number, N> &mat);`
- `LaMatrix<number, N> operator-(const LaMatrix<number, N> &mat) const;`
- `void operator-=(const LaMatrix<number, N> &mat);`
- `LaVector<number, N> operator*(const LaVector<number, N> &vec) const;`
- `LaMatrix<number, N> operator*(number num) const;`
- `void operator*=(number num);`
- `LaMatrix<number, N> operator/(number num) const;`
- `void operator/=(number num);`

You can approach the vectorization of these member functions in a similar way as for the linear algebra vector class. If it helps, start by implementing the core functionality using vector operations, and handle the remainder—the elements that don't fit into a complete vector unit—with a scalar implementation. Once your code works correctly, you can proceed to vectorize the remaining scalar parts.

Hint: Generally, it's a good idea to vectorize the innermost loop for optimal performance.

Vectorized Computation of the Trace

In the previous task, we explicitly avoided vectorizing the computation of the trace. In fact, with the current memory layout used in our `LaMatrix` class, a vectorized implementation of the trace is not likely to yield performance benefits. Explain why vectorizing this computation is difficult, and why—despite being theoretically possible—it is not worth the effort from a performance perspective.

If you're unsure why this is the case, the best way to find out might be to try vectorizing the trace computation yourself. While working on the implementation, you may gain insight into why the performance benefit is minimal, or why the vectorized version ends up being more complex than expected.

Hint: Vectorized operations typically require a contiguous memory layout to be efficient.

Vectorized Computation of Matrix-Matrix Product

So far, we have not vectorized the implementation of the matrix-matrix product. The reason for this is simple: memory access patterns present a challenge.

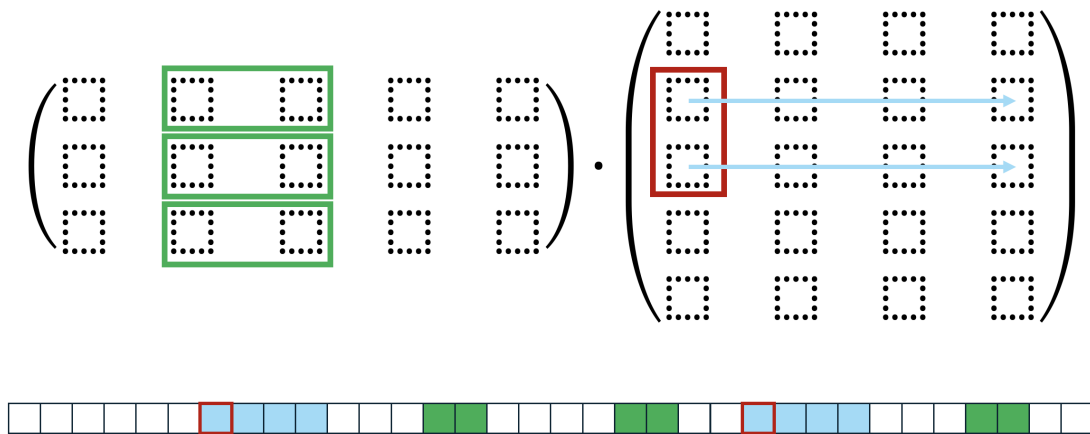


Figure 1: Illustration of the memory access problem during vectorized matrix-matrix multiplication. The top shows an example where the green blocks represent (vectorized) chunks of entries to be multiplied by the red block in matrix B . The blue arrows indicate memory storage direction. The bottom shows how these blocks are laid out in memory: green blocks are contiguous and efficient to load, while red blocks are scattered, requiring costly memory loads to fill vector registers.

Assume we want to compute $C = A \cdot B$, where $A \in \mathbb{R}^{M \times N}$, $B \in \mathbb{R}^{N \times P}$, and therefore $C \in \mathbb{R}^{M \times P}$. To vectorize the operation, we typically process matrix A **row-wise** and matrix B **column-wise**, as illustrated in Figure 1.

The core issue lies in how the matrices are stored in memory. In our implementation we use `std::vector<std::vector<number>>`, which stores each row as a contiguous block of memory. This means loading parts of a row from matrix A into vector registers is efficient. However, columns in matrix B are *not* stored contiguously—they are scattered across memory. Accessing them incurs a significant performance penalty due to inefficient memory loads.

There are various strategies to improve this situation, but we will use the simplest one: **we aim to minimize memory access to matrix B** , which suffers the most from the non-contiguous layout. In our implementation, the goal is to **access each vectorized block of matrix B only once**, even if this means accessing blocks in matrices A and C multiple times. This approach helps reduce the expensive memory operations that would otherwise result from repeatedly reading scattered column data in B .

Your task is to implement this!

Bonus Question (+2 Points):

In this context, which combinations of cache line size and vectorization width are favorable for performance? Conversely, for which hardware configurations might this approach become inefficient—suggesting that a different algorithm may be more suitable for implementing a fast matrix-matrix product?

Task 3: Putting it All Together – The Heat Operator (8 Points)

Having vectorized both the vector and matrix implementations, we now turn our attention to the matrix-based heat operator. This task does not require any new implementation. Instead, similar to Task 1, the focus is on analyzing the performance of the matrix-based heat operator code.

To support your analysis, a benchmarking framework has been prepared for you and is located in: `benchmarks/heat/heat_operator_benchmark.cpp`. Run this benchmark and examine the reported runtimes. Pay particular attention to the effects of using different data types and vectorization widths. What trends do you observe? Do these trends align with those you saw in Task 1 during the performance analysis of your vector implementation? To gain further insight, an additional benchmark is available in `benchmarks/linear_algebra/la_matrix_benchmark.cpp`.

This benchmark tests your implementation of the matrix-vector product from Task 2, which is a core component of the matrix-based heat operator. Reviewing its performance may help explain the behavior observed in the heat operator benchmark.

As before, be aware that the observed results may not always align with expectations due to the experimental nature of the `std::experimental::simd` implementation. This is completely normal. Focus instead on identifying trends, comparing them with earlier observations, and reasoning about the causes. Which trends match your expectations? Which do not?

Important: In your report, please include the full benchmark results from your local machine. This is essential for understanding and verifying your interpretation of the trends.